

实验三：对称密码算法的软件实现

一、实验目标

从基本实现出发，优化对称密码（SM4 / AES / GIFT / TWINE）的软件执行效率，覆盖 T-table、shuffle 以及最新的指令集中的两种方法基于加解密软件实现，做 CTR / GCM / XTS 工作模式的软件优化实现。（本篇实验报告内选择 SM4 与 CTR 实现为例）

二、实验原理

2.1 SM4

SM4 是中国国家密码管理局发布的分组密码标准，分组长度为 128 位（16 字节），密钥长度为 128 位。算法采用 32 轮非线性迭代结构，每轮使用一个 32 位轮密钥，核心变换：

S 盒替换：8 位输入→8 位输出，非线性变换

线性变换 L： $L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$

轮函数： $X[i+4] = X[i] \oplus L(S(X[i+1] \oplus X[i+2] \oplus X[i+3] \oplus RK[i]))$

2.2 优化策略原理

1. T-Table 查表优化：将 S 盒替换和线性变换 L 合并为 4 张查找表（T-Box），每张表 256 个条目，每个条目 32 位，一轮运算从 4 次 S 盒查表+线性变换简化为 4 次 T-Box 查表+3 次异或，减少了计算量
2. SIMD 向量化优化（SSSE3）：使用 SSSE3 指令集的 `_mm_shuffle_epi8` 指令，一次处理 16 个字节的 S 盒替换。将 16 个字节同时加载到 128 位寄存器，利用单指令多数据（SIMD）并行执行 S 盒查找，理论上可达到 16 倍并行度。实现原理：

将 S 盒拆分为高 4 位和低 4 位两张查找表,使用 `_mm_shuffle_epi8` 分别查表后异或组合。。

3. GCM 模式设计思想: GCM (Galois/Counter Mode) 结合了 CTR 模式的并行加密能力和 GHASH 认证功能。其优化关键在于: CTR 部分使用 T-Table/SIMD 优化; GHASH 部分使用预计算表加速有限域乘法。

4. XTS 模式设计思想: XTS (XEX-based Tweaked-codebook mode with ciphertext Stealing) 主要用于磁盘加密,使用两个密钥: 一个用于数据加密,一个用于调整值 (Tweak) 加密。其优化关键: 预计算 Tweak 的加密结果; 利用 CTR 模式的并行特性处理同一数据单元内的多个块。

2.3 工作模式

模式	并行性	认证	适用场景
CTR	高 (可并行)	无	通用加密
GCM	高 (可并行)	有	需要认证的通信
XTS	中 (数据单元内可并行)	无	存储加密

三、实验步骤与实验结果

3.1 SM4 实现

首先完成基础实现,完成异或、S 盒、线性变换等步骤

3.2 T-Table 优化实现

将 S 盒替换和线性变换 L 合并为 4 张预计算表:

```
void sm4_init_tbox(void) {  
  
    for (int i = 0; i < 256; i++) {  
  
        uint32_t s = SBOX[i];  
  
        T_BOX0[i] = sm4_l(s << 24);  

```

```

T_BOX1[i] = sm4_l(s << 16);

T_BOX2[i] = sm4_l(s << 8);

T_BOX3[i] = sm4_l(s);

}

```

一轮运算简化为：

```

uint32_t temp = x[i + 1] ^ x[i + 2] ^ x[i + 3] ^ rkey[i];

uint32_t t = T_BOX0[(temp >> 24) & 0xFF] ^

               T_BOX1[(temp >> 16) & 0xFF] ^

               T_BOX2[(temp >> 8) & 0xFF] ^

               T_BOX3[temp & 0xFF];

x[i + 4] = x[i] ^ t;

```

具体最终代码实现见附件

3.3 SIMD 向量化实现（SSSE3）

利用 SSSE3 的 `_mm_shuffle_epi8` 指令实现 16 字节并行的 S 盒替换。将 S 盒拆分为高 4 位和低 4 位两个查找表：

```

static inline __m128i sm4_sbox_ssse3(__m128i v) {

    // 分离低 4 位和高 4 位

    __m128i lo = _mm_and_si128(v, mask_low);

    __m128i hi = _mm_and_si128(_mm_srli_epi16(v, 4), mask_low);

    // 分别查表

    __m128i r_lo = _mm_shuffle_epi8(lut_lo, lo);

```

```

    __m128i r_hi = _mm_shuffle_epi8(lut_hi, hi);

    // 组合结果

    return _mm_xor_si128(r_lo, r_hi);

}

```

具体最终代码实现见附件

3.4 CTR 工作模式实现

CTR 模式将计数器加密后与明文异或，提高了并行性：

```

void sm4_ctr_encrypt(const uint8_t* in, uint8_t* out, size_t len,

    const uint8_t key[16], uint8_t iv[16],

    sm4_encrypt_fn encrypt_func) {

    // 密钥扩展

    uint32_t rkey[32];

    sm4_key_schedule(key, rkey);

    uint8_t ctr[16];

    memcpy(ctr, iv, 16);

    for (每个数据块) {

        encrypt_func(ctr, keystream, rkey); // 加密计数器

        XOR(明文, 密钥流, 密文);           // 异或

        inc_counter(ctr);                   // 计数器加 1

    }

}

```

具体最终代码实现见附件

四、实验结果

代码运行结果如图

```
=====
SM4-CTR 软件实现优化与性能对比测试（数据量：16 MB）
=====

[1] 基础实现 (Basic Implementations)
    正确性验证：通过 (PASS)
    运行耗时：   513.15 ms
    吞吐率：     31.18 MB/s

[2] T-Box 查表优化 (T-Table Optimization)
    正确性验证：通过 (PASS)
    运行耗时：   367.39 ms
    吞吐率：     43.55 MB/s
    相对加速比： 1.40x

[3] Shuffle 向量化优化 (SIMD Optimization)
    正确性验证：通过 (PASS)
    运行耗时：   374.28 ms
    吞吐率：     42.75 MB/s
    相对加速比： 1.37x
=====
```